JPA. JQL Java Persistence Query Language

Introducción JPA

JPA es un componente de la arquitectura J2EE que permite establecer un interface de mapeo ORM (Object Relational Mapping) para gestionar datos en aplicaciones Java.

Los datos que se gestionan en Java normalmente se solucionan a través de objetos, que tienen sus propios estados y comportamientos. Sin embargo, a la hora de almacenarlos (persistirlos) se usa habitualmente una base de datos relacional. Es por tanto necesario mapear o convertir esos objetos de Java en filas dentro de una tabla relacional.

A lo largo del tiempo se han intentado varias soluciones, tanto particulares como orientadas a un estándar, pero normalmente eran complejas de usar y sin una continuidad asegurada.

Para eso nació JPA, que se convierte en el estándar a la hora de realizar esta conversión objeto-relacional. Es una capa que se implementa sobre JDBC y que realiza por nosotros todo el trabajo pesado de mapeo. Este mapeo se suele hacer sobre metadatos, como XML o anotaciones.

De manera general, JPA viene implementado en varias áreas:

- Las API que permiten la persistencia en Java
- Un lenguaje de consulta denominado JPQL
- Los metadatos que permiten el mapeo entre objetos y tablas relacionales

La funcionalidad de JPA es implementada a través de un proveedor de persistencia de los que existen en la actualidad (como Hibernate, Eclipse Link, etc). Una de las ventajas de JPA es la de poder elegir en cualquier momento el proveedor que más se adapte a nuestras necesidades. Por tanto, es el proveedor quién implementa y realiza el trabajo de persistencia, pero siempre de acuerdo a las especificaciones indicadas en las API de JPA.

Durante este artículo vamos a centrarnos en el lenguaje de consulta **JPQL (Java Persistence Query Language)**, que permite acceder de forma rápida y sencilla a los datos que almacenamos a través de JPA.

Es conveniente, aunque no obligatorio conocer un poco el lenguaje SQL para sacar el mayor partido a JPQL, ya que muchas de sus instrucciones son iguales o muy parecidas.

Introducción a JPQL

Usando la interface “EntityManager”, podemos realizar accesos a la Base de datos, a través de la entidad correspondiente, pero tiene bastantes limitaciones en su funcionalidad.

Al contrario, JPQL nos permite realizar consultas a través de multitud de criterios posibles y obtener más de un objeto por consulta.

Es por tanto un entorno mucho más potente y sencillo de utilizar que comandos Java etándar.

Consultas. SELECT

Para poder acceder a los datos almacenados en una tabla relacional, la forma más simple de consulta JPQL es la siguiente

```
SELECT a FROM alumnos a
```

A través de este comando, podemos recuperar todos los objetos de la clase “Alumnos” que estarán almacenadas en la Base de Datos. Como vemos, el comando es muy similar a la SELECT del lenguaje SQL.

Como diferencia, la segunda cláusula “a” es un alias de la clase Alumnos que es usada por la primera “a” (denominada expresión) para acceder a la tabla de la Base de Datos y a sus propiedades (columnas de la tabla).

Por ejemplo para acceder a un atributo o columna de la tabla.

```
SELECT a.nombre FROM alumnos a
```

Por tanto, al igual que en SQL, desde JPQL se utiliza la notación de puntos para acceder a las propiedades.

```
SELECT a.nombre, a.apellidos FROM Alumnos a
```

Las consulta JPQL devuelve de 0 a N filas en cada operación, que luego se tendrán que tratar

Como SQL, se puede usar la cláusula `DISTINCT` para devolver filas que no estén duplicadas:

```
SELECT DISCTINCT a.nombre FROM Alumnos a
```

También podemos usar funciones dentro de las consultas. De nuevo vemos el parecido con SQL, ya que se usa la función COUNT para devolver el número de filas existentes en la tabla.

```
SELECT COUNT(a) FROM Alumnos a
```

Condiciones. Cláusula WHERE

Mediante la cláusula WHERE podemos poner condiciones en las consultas. De esta forma restringimos el número de filas a devolver por la consulta SELECT.

Su formato es el siguiente. En este caso recuperamos todos los alumnos que hayan suspendido matemáticas, suponiendo que haya un atributo que tenga ese dato (nota_matematicas).

```
SELECT a FROM Alumnos a
WHERE a.nota_matematicas < 5
```

Por supueto, podemos realizar consultas complejas y combinadas, utilizando los operadores condicionales AND, OR, etc… En el siguiente ejemplo vemos los alumnos que hayan suspendido matemáticas y lengua.

```
SELECT a FROM Alumnos a
WHERE a.nota_matematicas < 5 AND a.nota_lengua < 5
```

Otro ejemplo, en este caso de negación, vemos aquellos alumnos que hayan suspendido matemáticas pero no lengua.

```
SELECT a FROM Alumnos a
WHERE a.nota_matematicas < 5 AND NOT (a.nota_lengua < 5)
```

Otro operador que se puede utilizar es “BETWEEN” que permite buscar por un rango. Por ejemplo, si queremos saber los alumnos que hayan sacado 9 o 10.

```
SELECT a FROM Alumnos a
WHERE a.nota BETWEEN 9 AND 10
```

También es posible utilizar el operador LIKE que permite recuperar filas en columnas de tipo carácter. El siguiente ejemplo recupera todos los alumnos cuyo nombre comience por “AL”, por ejemplo Alberto o Alberto.

```
SELECT a FROM Alumnos a
WHERE a.nombre LIKE 'AL%'
```

Se utilizan comodines como “%” que sustituye un conjunto de caracteres o “_” que sustituye un solo carácter. De nuevo, el conocimiento de SQL es vital para poder sacarle partido a JPQL.

Existen otros operadores, funciones y componentes que se pueden consultar en el manual de referencia de JPQL.

Ordenar resultados. Cláusula Order by

La cláusula “Order by” permite realizar una ordenación de los datos devueltos. Con la cláusula `ASC`, lo hacemos de forma ascendente y con DESC de forma descendiente.

```
SELECT a FROM Alumnos a
ORDER BY a.nombre DESC
```

Otro ejemplo, en este caso con una ordenación combinada.

```
SELECT a FROM Alumnos a
ORDER BY a.nombre DESC, a.apellidos ASC
```

Modificar objetos. UPDATE

Para modificar las instancias de objetos que se encuentren dentro de la tabla alumnos se usa la cláusula UPDATE. También es muy similar a la que se utiliza en el lenguaje SQL.

```
UPDATE Alumnos  a
SET a.nota_matematicas = 5
WHERE a.cod_alumno = 1001
```

En este caso modificamos la nota de matemáticas del alumno cuyo código es el 1001. Vemos que se puede utilizar la cláusula WHERE para acotar las filas modificadas, de lo contrario se cambiarían todos los objetos de la tabla.

Borrados de fila. DELETE

EL comando para borrar filas es DELETE. También se utiliza con la cláusula WHERE para acotar las filas que se deben borrar.

```
DELETE FROM Alumnos a
WHERE a.alumno = 1001
```

Ejecutar sentencias JQPL

Una vez que hemos visto como montar y crear sentencias JPQL llega el momento de lanzarlas. Dado que lo tenemos que hacer dentro de un programa Java es necesario usar una serie de métodos y pasos para poder hacerlo.

El lenguaje JPQL se integra implementado la interface “Query”. Esta implementación se obtiene a través de “EntityManager”. Se usan diversos métodos de factoría, entre los que destacan los siguientes:

- createQuery(String jpql)
- createNamedQuery(String name)
- createNativeQuery(String sql)

Vamos a crer como crear un objeto “Query” y realizar una consulta a la base de datos. En este ejemplo ignoramos los métodos que no tienen que ver con JPQL propiamente dicho.

```
package es.cursojpa.cle;

import java.util.List;

import javax.persistence.EntityManager;
import javax.persistence.EntityManagerFactory;
import javax.persistence.EntityTransaction;
```

```
import javax.persistence.Persistence;
import javax.persistence.Query;

public class Main {

    public static void main(String[] args) {
        EntityManagerFactory emf = Persistence
                .createEntityManagerFactory("JPA_Factoria");
        EntityManager em = emf.createEntityManager();

        String jpql = "SELECT a FROM Alumnos a";
```

//Creamos un query a través del método createQuery

```
  Query query = em.createQuery(jpql);

        List<Alumno> resultados = query.getResultList();

        for(Alumno a : resultados)

 {
            //Comandos...
        }

        em.close();
        emf.close();
    }
}
```

Creamos un Query a través del método createQuery(String) del propio EntityManager. La sentencia JPQL se pasa en modo de cadena de texto.

Una vez obtenido el objeto Query podemos invocar al método retResultList() para recuperar mediante una lista todas las instancias de Alumno que se recuperen a través de la SELECT.

Después podemos realizar las operaciones sobre los datos que estimemos oportunas, ya desde comandos Java

Parámetro Dinámicos

Una de las características más potente de JPQL es la posibilidad de poner parámetros dinámicos, es decir pasar valores que pueden variar cada ve que se ejecuta la consulta.

Se realiza la asignación a través de la posición o del nombre del parámetro.

- Para usar la sintaxis de tipo posición se usa “?posicion”. Por ejemplo “?1”
- Para usar la notación por nombre se usa la sintaxis (:nombre).

Por ejemplo en el caso de pasar un parámetro dinámico por posición:

```
SELECT a FROM Alumnos a
WHERE a.nombre = ?1
```

En el caso de acceder por nombre

```
SELECT a FROM Alumnos a
WHERE a.nombre = :nombre
```

Los valores de estos parámetros se pasan en el momento de ejecutar la consulta. Usamos el método "setPArameter" para incluir un parámetro de forma dinámica en la consulta. Por ejemplo para pedir aquellos alumnos cuya nota en matemáticas sea superior a 5 y cuyo nombre comience por "AL" realizamos el siguiente código, en este caso pasando los valores por posición.

```
String jpql = "SELECT a FROM Alumnos a WHERE a.nota_matematicas > ?1
AND a.nombre like = ?2"
Query query = em.createQuery(jpql);
query.setParameter(1, 5);
query.setParameter(2, "AL%");
List<Pelicula> resultados = query.getResultList();
```

Si el valor que pasamos como segundo argumento no se corresponde con el valor esperado (por ejemplo, una cadena de texto donde se espera un valor numérico), se dispara la excepción `IllegalArgumentException`. Esto también sucede si intentamos asignar un valor a un parámetro dinámico que no existe.

En el caso de querer utilizar la nomenclatura por nombre usamos el siguiente formato:

```
String jpql = "SELECT a FROM Alumnos a WHERE a.nota_matematicas >
:nota AND a.nombre like :nombre"
Query query = em.createQuery(jpql);
query.setParameter("nota", 5);
query.setParameter("nombre", "AL%");
List<Pelicula> resultados = query.getResultList();
```

Consultas estáticas

Este tipo de consultas son un poco distintas a las anteriores, que son dinámicas. La diferencia básica es que una vez definidas no pueden ser modificadas, es decir se graban a fuego dentro del programa.

Por tanto solo se leen y cargan cuando el programa arranca, no cada vez que son ejecutadas. Al estar previamente "compiladas" son más eficientes que las dinámicas y disponen de un mejor rendimiento.

Estas consultas se definen mediante metadatos XML o anotaciones. Por ejemplo:

```
@Entity
@NamedQuery(name="verAlumnos", query="SELECT a FROM Alumnos a")
public class Alumno { ... }
```

En este caso se usa la anotación @NamedQuery para diseñar la consulta. La ponemos un nombre y una cadena con la sentencia JPQL que hay que ejecutar. El nombre que usemos para la consulta debe ser único en toda la unidad de persistencia. Como vemos la creamos en la Entity correspondiente, en este calso Alumno.

Una vez que hemos definido la consulta podemos utilizarla a través del objeto

```
Query query = em.createNamedQuery(Alumno.verAlumnos);
List<Alumno> resultados = query.getResultList();
```

Consultas Nativas de SQL

Otra forma de usar comandos SQL es la de hacerlo de forma nativa, en vez de JPQL. Es decir, lanzamos un comando SQL contra la Base de Datos. De esta forma y en algunas ocasiones podemos aprovecharnos de alguna característica del motor de Base de Datos que estemos usando, por ejemplo Oracle y de esa forma mejorar el rendimiento.

Como efecto negativo, tenemos que un comando nativo en una Base de Datos puede no funcionar correctamente en otra y por tanto tenemos que ser cuidadosos con su uso.

Para realizar esta operación usamos le método "createNativeQuery", lanzando la consulta como una cadena.

Por ejemplo:

```
String sql = "SELECT * FROM Alumnos";
Query query = em.createNativeQuery(sql);
...
```

También podemos definirlas de forma estática:

```
@Entity
@NamedNativeQuery(name=Alumno.ver_alumnos, query="SELECT * FROM
alumnos")
public class Alumno {
}
```

19
sep 2013
POSTED BY Elena
POSTED IN Tic-tek
DISCUSSION 0 Comments
TAGS

J2EE, Java, Jpa, jpql, ORm

Originalmente en: http://tic-tek.es/jpa-jql-java-persistence-query-language/